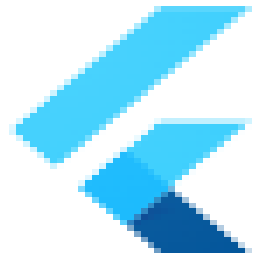


المحاضرة السادسة: تصنيفات أدوات واجهات التطبيقات في Flutter



Flutter

م. سليمان الشوصي

❖ التصنيفات الأساسية للـ Widgets

١. الحاويات المرئية (Visual Container Widgets) هي عناصر تحتضن Widgets أخرى ولها مظهر بصري واضح (ألوان، حواف، ظلال).

• الأدوات: مثل AppBar | Card | Container

٢. العناصر الأساسية والوسائط (Primitive & Media Widgets): هي عناصر قائمة بذاتها تستقبل خصائص وقيم مباشرة (لا تحتوى عناصر أخرى).

• الأدوات: مثل Icon | Image | Text

٣. عناصر الهيكل والتنسيق (Invisible Layout Widgets): عناصر تستقبل Widgets أخرى وتنظم موقعها، ليس لها مظهر بصري مرئي.

• الأدوات: مثل Flexible | Expanded | Center | Padding | Column | Row | Scaffold.

٤. أزرار التفاعل والإدخال (Interactive Widgets) هي عناصر تستقبل Widgets وتتفاعل مع ضغطات ومدخلات المستخدم..

• الأدوات: مثل MaterialButton | TextButton | ElevatedButton .

```
Card(
  elevation: 20,
  color: const Color.fromARGB(255, 240, 193, 248),
  shadowColor: Colors.green,
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(15), // Ro
  ),
  margin: EdgeInsets.all(10),
  child: Padding(padding: const EdgeInsets.all(25),
    child: Text("عنصر حاوي: Card"),
  ), // Padding // Card
```



❖ **مفهوم Card:** هو عنصر حاوي (Container) يعتمد على لغة تصميم Material Design، يُستخدم لتجميع المعلومات والبيانات المرتبطة ببعضها (مثل صورة، عنوان، نص) داخل حاوية واحدة ذات حواف دائرية وظل خفيف، مما يمنح التطبيق مظهر تنظيمي جذاب وبُعداً بصرياً على الشاشة.

أهم الخصائص:

elevation: يحدد ارتفاع الكارت عن الشاشة ومقدار بروز الظل خلفه.

color: يحدد لون خلفية الكارت.

shadowColor: يحدد لون الظل الممتد أسفل الكارت.

margin: يحدد المسافة الخارجية بين الكارت والعناصر المحيطة به.

shape: يحدد شكل الحواف والزوايا (مثل جعلها دائرية باستخدام

(BorderRadius

child: العنصر الابن أو الواجهة المعروضة داخل الكارت.

```
main.dart pubspec.yaml
pubspec.yaml
49 flutter:
50   uses-material-design: true
51   assets:
52     - assets/images/
53     # - images/a_dot_ham.jpeg
```

```
Container(width: 400, height: 200,
  color: Colors.red,
  child: Image.asset(
    'assets/images/quran2.JPG',
    fit: BoxFit.contain,
    repeat: ImageRepeat.repeatX,
  ), // Image.asset // Container
```

```
Expanded(
  child: Image.network(
    'https://storage.ghost.io/c/56/e6/56e6bee2-64b3-46fc-9f38-839cefa57dd7/content/images/size/w2000/2023/07/Flutter-feature.png',
    width: 200,
    height: 200,
    fit: BoxFit.contain,
    repeat: ImageRepeat.repeatY,
  ), // Image.network
), // Expanded
```

٣. أداة **Image.asset** و **Image.network**: أدوات متخصصة لعرض الصور؛ الأولى من الملفات المحلية للمشروع والثانية مباشرة عبر رابط شبكة إنترنت.

❖ **ملاحظة**: عند التعامل مع أداة **image.asset** في المجلدات و ثم تضمين مسار المجلد الى ملف **pubspec.yaml** الخاص بتحميل المكاتب كما هو موضح في الصورة الثانية

أهم الخصائص: **name / src**: مسار الصورة المحلي أو رابط ال URL.

- width / height**: أبعاد العرض والارتفاع المخصصين للصورة.

- Fit**: تُحدد خوارزمية تمدد واحتواء الصورة داخل إطارها عبر **BoxFit** (مثل **cover**, **contain**, **fill**).

١. **BoxFit.cover**: تقوم بتكبير الصورة لتغطي كامل المساحة المتاحة للمغلف (**Container**) مع الحفاظ على نسبة الطول إلى العرض، وتُقص الأجزاء التي تخرج عن حدود الحاوية.

٢. **BoxFit.contain**: **تضمن** ظهور الصورة بالكامل داخل الحاوية دون قص، مع الحفاظ على نسبة أبعادها الأصلية (قد تترك مساحات فارغة في الجوانب).

٣. **BoxFit.fill**: تقوم بمط وتغيير أبعاد الصورة لتملأ الحاوية بالكامل في الاتجاهين الأفقي والعمودي، مما قد يسبب تشوهاً في مظهر الصورة الأصلي.

❖ **خاصية التكرار repeat**: تُستخدم هذه الخاصية لتحديد كيفية تكرار الصورة وتعبئتها للمساحة المتاحة عندما تكون أبعاد الحاوية أكبر من أبعاد الصورة الأصلية. تأخذ القيمة من نوع **Enum** يُسمى **ImageRepeat**.

- ImageRepeat.noRepeat** (الافتراضي): تعريض الصورة مرة واحدة فقط دون أي تكرار.

- ImageRepeat.repeat**: تكرار الصورة على كلا المحورين (الأفقي X والعمودي Y) حتى تملأ الحاوية بالكامل (تأثير النقوش أو البلاط).

- ImageRepeat.repeatX**: تكرار الصورة أفقياً فقط على المحور X.

- ImageRepeat.repeatY**: تكرار الصورة عمودياً فقط على المحور Y.



٤. أداة Column (التنسيق العمودي): تخطيط (Layout Widget) يقوم بترتيب العناصر بداخلها رأسياً فوق بعضها البعض.

أهم الخصائص:

```
Column(  
  // توزيع متساو للمسافة بين العناصر عمودياً  
  mainAxisAlignment: MainAxisAlignment.spaceAround,  
  // محاذاة العناصر لليسار أفقياً  
  crossAxisAlignment: CrossAxisAlignment.start,  
  children: <Widget>[  
    const Text('معلومات اخرى',  
      style: TextStyle(fontSize: 16)),  
    const SizedBox(height: 10),  
    ElevatedButton(onPressed: () {},  
      child: const Text('MyButton'))],  
  ) // <Widget>[] // Column
```

الخاصية (Property)	الوصف التفصيلي
children (List<Widget>)	قائمة الـ العناصر Widgets المرتبة أفقياً عمودياً.
mainAxisAlignment	كيفية توزيع الـ Widgets على طول المحور الرئيسي (الرئيسي). القيم الشائعة start : البداية، end : النهاية، center : الوسط، spaceAround : مسافات حول العناصر، spaceBetween : مسافات بين العناصر.
crossAxisAlignment	كيفية محاذاة الـ Widgets على طول المحور المتقاطع (المحور الافقي). القيم الشائعة start : الأعلى، end : الأسفل، center : الوسط، stretch : تمتد لملء الارتفاع
mainAxisSize	تحديد المساحة الرأسية التي يحجزها العمود. يحدد حجم الـ column على طول المحور الرئيسي (أفقياً). القيمة max تجعلها تملأ أكبر ارتفاع متاح لها القيمة البديلة هي min التي تجعلها تأخذ فقط المساحة التي تحتاجها العناصر.

٥. الحشو الداخلي **Padding**: هو Widget أبسط من Container، ووظيفته الوحيدة هي إضافة مساحة حشو داخلي Padding حول الـ Widget الابن.

٦. التوسيط **Center**: يعمل على توسيط الـ Widget الابن في منتصف المساحة المتاحة له من الـ Widget الأب.

٢. أداة Row (التنسيق الأفقي): تخطيط (Layout Widget) يقوم بترتيب العناصر الابن بداخلها بأسلوب أفقي متجاور (سطري).

صائص ROW	الوصف
children (List<Widget>)	قائمة الـ العناصر Widgets المرتبة أفقياً سيتم وضعها في الصف.
mainAxisAlignment	كيفية توزيع الـ Widgets على طول المحور الرئيسي (الأفقي). القيم الشائعة start: البداية، end: النهاية، center الوسط، spaceAround مسافات حول العناصر، spaceBetween مسافات بين العناصر.
crossAxisAlignment	كيفية محاذاة الـ Widgets على طول المحور المتقاطع (العمودي). القيم الشائعة start: الأعلى، end: الأسفل، center الوسط، stretch تتمدد لملء الارتفاع
mainAxisSize	MainAxisSize.max: يحدد حجم الـ Row على طول المحور الرئيسي (أفقياً). القيمة max تجعلها تملأ أكبر عرض متاح لها، القيمة البديلة هي min التي تجعلها تأخذ فقط المساحة التي تحتاجها العناصر.

أهم الخصائص:

```
Row( //max يجعل الـ
  mainAxisAlignment: MainAxisAlignment.max,
  // توزيع متساو للمسافة بين العناصر
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  // محاذاة العناصر للأسفل عمودياً
  crossAxisAlignment: CrossAxisAlignment.end,
  children: <Widget>[
    const Icon(Icons.thumb_up),
    const Icon(Icons.thumb_up),
    const Icon(Icons.thumb_up),
  ]), // <Widget>[] // Row
```

```
child: Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: <Widget>[
    Expanded(flex: 5,
      child: Text('You have Widget Expanded:')), /
    Expanded(flex: 1,
      child: Text('You have Widget Expanded:')),
```

```
Container(height: 200,width: 400,
  padding: EdgeInsets.all(10),
  color: Colors.red,
  child: Flexible(fit: FlexFit.tight,
    child: Container(color: Colors.blue,
      child: Text("Flexible_Widget"),
    )), // Container // Flexible // Cont
```

المحاضرة السادسة

Flexible_Widget

أداة Expanded: أداة مُغلّفة تجعل العنصر يأخذ كل المساحة الإضافية المتبقية في الـ Row. هذا يضمن توزيع المساحة بشكل ديناميكي بين العناصر المرنة وغير المرنة.

أداة Flexible: أداة مُغلّفة تسمح للعنصر بالمرونة في الحجم ولكن لا تجعله ملزماً بأخذ كل المساحة المتبقية **fit** يجعل سلوكه مشابهاً لـ **loose** يسمح للعنصر بأخذ المساحة الإضافية لكن بحجمه الطبيعي أو أقل.

تُصنّف الأدوات Expanded و Flexible ضمن الفئة الثانية: عناصر الهيكل والتنسيق (Invisible Layout Widgets).

سبب التصنيف:

ليس لها مظهر بصري مرئي: لا تملك لوناً، ظلاً، أو حوافاً تظهر للمستخدم على الشاشة. تستقبل كائنات داخلها **(child)**: وظيفتها الأساسية هي التفاف عنصر آخر **(child)** للتحكم في كيفية استهلاكه للمساحة المتاحة.

تُستخدم حصراً داخل عناصر التنسيق: تعمل فقط داخل Row أو Column أو Flex للتحكم في توزيع المساحة بين العناصر وتجنب مشكلة المفيض البصري (Overflow Error).

الفرق بينهما :

Expanded: تجبر العنصر الداخلي على التمدد ليملأ كل المساحة المتبقية في الصف أو العمود.

Flexible: تعطي العنصر المرونة ليأخذ جزءاً من المساحة المتبقية فقط بقدر حاجته، دون إجباره على ملئها بالكامل

```
//-----ElevatedButton-----
ElevatedButton(
  onPressed: () {
    print("SULIMAN_ALSHOSSI");},
  style: ElevatedButton.styleFrom(
    elevation: 20,
    shadowColor: Colors.green,
    backgroundColor: Colors.blue,
    padding: EdgeInsets.all(30),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(10),
    ), // RoundedRectangleBorder
    textStyle: TextStyle(fontSize: 15),),
  child: Text("button")), // ElevatedButton

//-----ElevatedButton.icon----
ElevatedButton.icon( onPressed: () {},
  label: Text("button_ICON"),
  icon: Icon(Icons.home),
  style: ElevatedButton.styleFrom(
    elevation: 20,
    shadowColor: Colors.green,
    backgroundColor: Colors.blue,
    padding: EdgeInsets.all(30),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(10),
    ), // RoundedRectangleBorder
    textStyle: TextStyle(fontSize: 15), ),), /
```

أداة **ElevatedButton**: زر بارز (مرفوع) في Flutter يُستخدم لتنفيذ إجراء معين عند النقر عليه، وله ظل خلفي (Elevation) يمنحه إحساساً بالعمق والبروز فوق واجهة المستخدم.

أهم الخصائص: `padding` | `shape` | `backgroundColor` | `textStyle`.

- **onPressed**: دالة التفاعل التي تُنفَّذ الكود المحدد (طباعة الاسم) عند النقر على الزر.
- **elevation**: يحدد ارتفاع الزر عن الشاشة ومقدار بروز الظل (بقيمة ٢٠).
- **shadowColor**: يحدد لون الظل الناتج عن ارتفاع الزر (الأخضر).
- **child**: العنصر الداخلي المعروض داخل الزر (عنصر نصي `Text`).

أداة **ElevatedButton.icon**: هو نوع خاص من `ElevatedButton` يُستخدم لخلق زر بارز يدمج بين أيقونة ونص معاً بشكل برمجي جاهز ومُنظم تلقائياً (تكون الأيقونة على اليسار والنص على اليمين بشكل افتراضي).

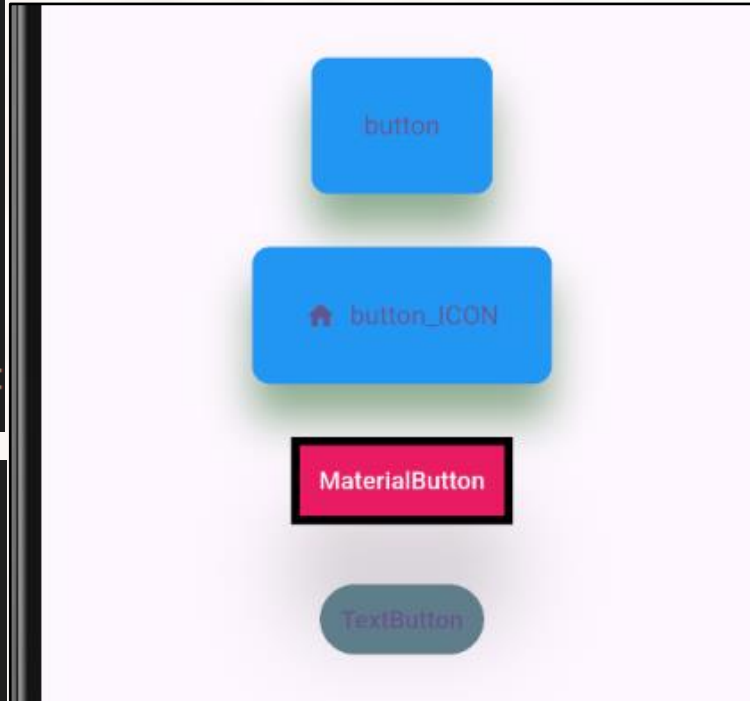
الخصائص الجديدة والمختلفة عن الزر العادي:

- **icon**: الخاصية المسؤولة عن تحديد الأيقونة المعروضة داخل الزر (في الكود: أيقونة المنزل `Icons.home`).
- **label**: بديل لخاصية **child** في الزر العادي، وتُستخدم لتحديد النص أو العنصر المعروض بجانب الأيقونة.
- باقي الخصائص مثل خصائص الزر السابق

```
//-----MaterialButton-----
MaterialButton( onPressed: () {},
  child: Text("MaterialButton"),
  elevation: 50,
  // يظهر لون الزر اثناء الضغط
  splashColor: Colors.green,
  color: Colors.pink,
  shape: Border.all(
    width: 5, color: Colors.black),
  textColor: Colors.white,
  height: 50, minWidth: 50, ), // Mat
```

```
//-----textButton-----
TextButton( onPressed: () {},
  child: Text("TextButton"),
  style: TextButton.styleFrom(
    backgroundColor: Colors.blueGrey)),
```

أداة **MaterialButton**: زر تقليدي يعتمد على تصميم Material Design الأساسي في Flutter، ويتميز بسهولة التخصيص المباشر للخصائص (مثل الألوان والأبعاد) دون الحاجة لاستخدام دالة `styleFrom`.



- أهم الخصائص:** | `elevation` | `shape` | `onPressed` | `child`
- color**: يحدد لون خلفية الزر الأساسية.
 - elevation**: يحدد ارتفاع الزر عن الشاشة ومقدار بروز الظل (بقيمة ٢٠).
 - splashColor**: يحدد اللون الذي يظهر كتأثير تموج عند الضغط على الزر (الأخضر).
 - height & minWidth**: يحددان الأبعاد المباشرة للزر (الارتفاع وأقل عرض ممكن).
 - textColor**: يحدد لون النص الداخلي مباشرة.

أداة **TextButton**: زر مسطح (Flat) وبسيط يُستخدم عادة للإجراءات الثانوية أو الضمنية (مثل الأزرار داخل الحوارات Dialogs أو بطاقات Card)؛ لا يحتوي على ظل أو خلفية بارزة افتراضياً، لكن يمكن تخصيص خلفيته ولونه حسب الحاجة.

الخصائص الجديدة والمختلفة عن الزر العادي: Style: يُستخدم لتنسيق الزر عبر `TextButton.styleFrom`:

الأداة **TextFormField** : هو Widget يُستخدم لإنشاء حقول إدخال نصية تسمح بـ التحقق من صحة البيانات (Validation) المدخلة.

خصائص المؤشر والتنسيق

الخاصية	الوصف الوظيفة
cursorColor	تُحدد لون المؤشر العلامة العمودية الواضحة
cursorHeight	تُحدد ارتفاع المؤشر بالبكسل.
cursorWidth	تُحدد عرض المؤشر بالبكسل.
style	تُحدد تنسيق النص المدخل فعلياً مثل اللون، الحجم، نوع الخط.
textAlign	تُحدد محاذاة النص داخل حقل الإدخال (في هذا المثال TextAlign.end: أي محاذاة لليمين).

خصائص التنسيق والديكور (InputDecoration) : ستخدم خصائص InputDecoration للتحكم في المظهر المحيط بحقل الإدخال.

الخاصية داخل InputDecoration	الوصف لوظيفتها
icon	أيقونة تظهر خارج حدود الحقل في جهة البداية.
border	تُحدد شكل ونمط حدود الحقل المحيطة بالنص.
suffixIcon	أيقونة ثابتة أو تفاعلية تظهر في أقصى الجهة اللاحقة داخل حدود الحقل.
labelText	نص العنوان، يظهر كعنوان عائِم فوق الحقل عند التركيز عليه.
labelStyle	تُحدد تنسيق النص المعنون.(labelText)
filled	تجعل خلفية الحقل مُملوءة باللون.
fillColor	تُحدد لون الخلفية عندما تكون خاصية filledمُفعلة.

خصائص الإدخال والتحكم (Input and Control)

الخاصية	الوصف الوظيفة
maxLength	تُحدد الحد الأقصى لعدد الأحرف المسموح بإدخالها.
maxLines	تُحدد الحد الأقصى لعدد الأسطر التي يمكن أن يصل إليها النص في حالة تعدد الأسطر.
keyboardType	تُحدد نوع لوحة المفاتيح التي تظهر للمستخدم (مثل لوحة الأرقام TextInputType.number).
textInputAction	تُحدد وظيفة زر "الإجراء" الذي يظهر في لوحة المفاتيح (مثل تحويل زر الإدخال إلى زر "بحث").(TextInputAction.search)
controller	عنصر تحكم (TextEditingController) يُستخدم للتحكم برمجياً في النص المدخل (مثل قراءة القيمة أو تغييرها).
obscureText	تُحدد ما إذا كان يجب إخفاء النص المدخل بنقاط أو رموز (تستخدم لكلمات السر).

Flutter Framework

```
TextFormField(  
  style: TextStyle(fontSize: 20, color: Colors.red),  
  maxLength: 10, maxLines: 2, minLines: 1,  
  // readOnly: true,  
  keyboardType: TextInputType.number,  
  textInputAction: TextInputAction.search,  
  cursorColor: Colors.red,  
  cursorHeight: 2, cursorWidth: 2,  
  // controller: password,  
  decoration: InputDecoration(icon: Icon(Icons.phone),  
    border: OutlineInputBorder(  
      borderRadius: BorderRadius.all(Radius.circular(10.0)),  
      borderSide: BorderSide(color: Colors.red, width: 10),  
    ), // OutlineInputBorder  
    hint: Text("data"),  
    filled: true,  
    fillColor: const Color.fromARGB(255, 227, 212, 167),  
    labelText: "ادخل البريد",  
    labelStyle: TextStyle(color: Colors.red, fontSize: 25),  
    suffix: Text("البريد"),  
    suffixIcon: Icon(Icons.email),  
  )), // InputDecoration // TextFormField
```



❖ المطلوب تنفيذ جميع الأدوات التي تم اخذها في هذه المحاضرة وتشكيلها في تطبيق واحد،

Thank you